TMM4128:
Machine Learning for Engineers

Andrei Lobov
e-mail: andrei.lobov@ntnu.no
phone: +47 45 913 455
room: 213, Verkstedteknisk
web: http://www.lobov.biz/academia

Slides:> http://lobov.biz/academia/mle/250213

Reference group

Reference groups - quality assurance of education

2-3 representatives for the class. Possible interface to give feedback on different matters related to the course.

Suggested 3 meetings, e.g. i) middle of February; ii) mid of March; and iii) end of April

Who wants / can be in the group?:
1) Elisabella Hjelpdahl
2) …
3) …

Announcement (13.02.): Adjusting course plans

As the last lecture happens to be on 11.04., and not 2.5, the A3 is to be “lighter”.

“Portfolio” of assignments = A1+A2+A3/2, where A3/2 represents a smaller workload in comparison to A1 and A2.

Also, the submission deadline for A1 is shifted for 26.02.

Outline : Dimensionality reduction (code samples)

- Definition / Motivation
- The Curse of Dimensionality
- Main Approaches
  - Projection
  - Manifold Learning
- PCA : Principal Component Analysis
  - Preserving the Variance
  - Principal Components
  - Projecting Down to *d* Dimensions
  - Using Scikit-Learn
  - Explained Variance Ratio
  - Choosing the Right Number of Dimensions
  - PCA for Compression
  - Randomized PCA
  - Incremental PCA
  - Kernel PCA
- LLE : Locally Linear Embedding
- Examples

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019 (Available online) / **Chapter 8**

Definitions (1)

Thousands or even millions of features for each training instance - **extremely slow**, **hard to find good solution** - the ***curse of dimensionality*.**

With the grow of number of features (dimensions), amount of samples needed to generalize grows exponentially.

Dimensionality reduction: reducing the number of features *considerably* to turn an intractable problem into a tractable one.

Definitions (2)

MNIST: 28*28 = 784 features.

* Some of the pixels are almost always white, so these can be dropped without losing much information.
* Neighboring pixels highly correlated - so, merging them into one (mean of the two intensities).

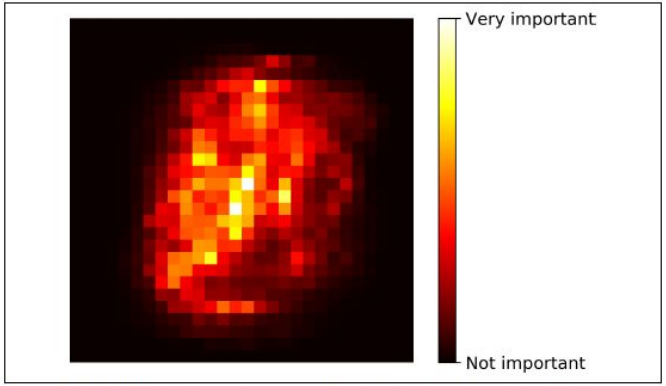

Figure 7-6. MNIST pixel importance (according to a Random Forest classifier)

The Curse of Dimensionality (1)

Going to high-dimensional space. Example: Tesseract

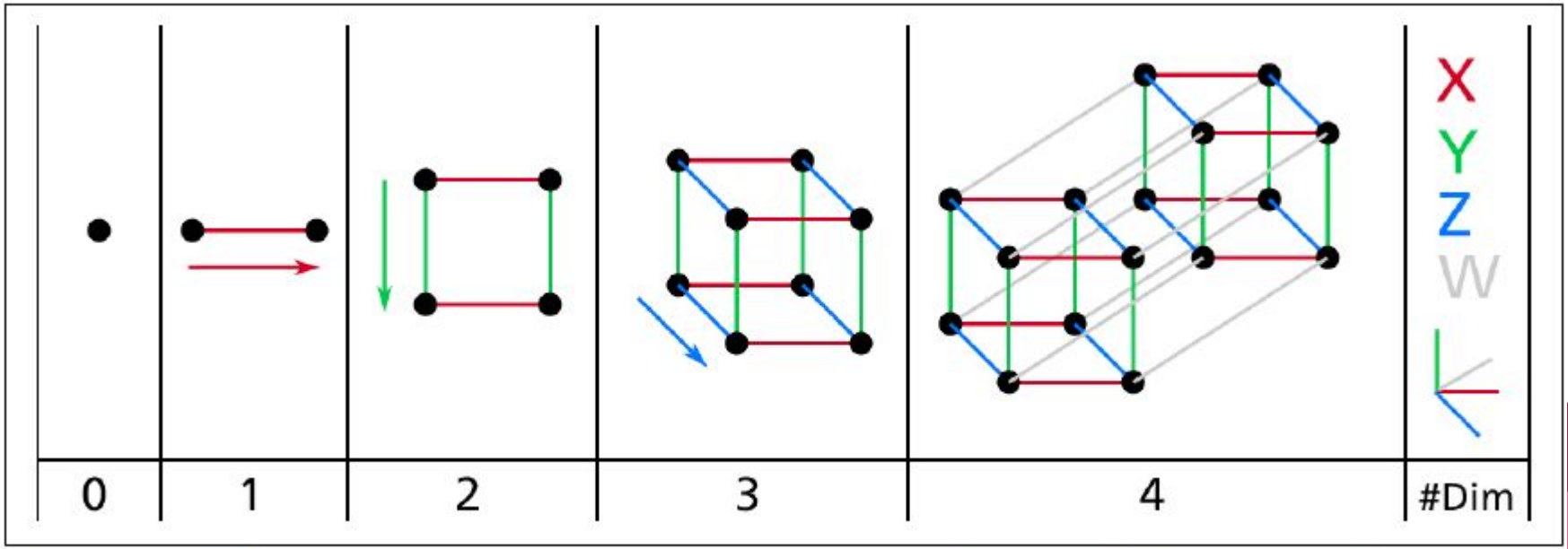

Figure 8-1. Point, segment, square, cube, and tesseract (0D to 4D hypercubes)[2]

The Curse of Dimensionality (2)

- Many things behave very differently in high-dimensional space.
- A random point in a unit square (a 1 × 1 square) has a 0.4% chance of being located less than 0.001 from a border.
- In a 10,000-dimensional unit hypercube (a 1 × 1 × ··· × 1 cube, with ten thousand 1s), this probability is greater than 99.999999%. Most points in a high-dimensional hypercube are very close to the border.
  Average distance between two points: 408.25 or $\sqrt{1,000,000/6}$

- 4D: $d((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$
- nD: $D_{ij}^2 = \sum_{v=1}^{n} (x_{vi} - x_{vj})^2$

The Curse of Dimensionality (3)

- High dimensional datasets are at risk of being very sparse: most training instances are likely to be far away from each other.
- New instance will likely be far away from any training instance, making predictions much less reliable than in lower dimensions.
- More dimensions the training set has, the greater the risk of overfitting it.
- Increasing the size of the training set to reach its sufficient density? → The number of training instances required to reach a given density grows exponentially with the number of dimensions.
- 100 features (<< features in MNIST dataset) → More training instances than atoms in the observable universe.

Main Approaches for Dimensionality Reduction

Projection
Manifold Learning

Projection (1)

- In most real-world problems, **training instances are not spread out *uniformly* across all dimensions**.
- Many features are almost constant, while others are highly correlated.
- All training instances actually lie within (or close to) a much lower-dimensional subspace of the high-dimensional space.

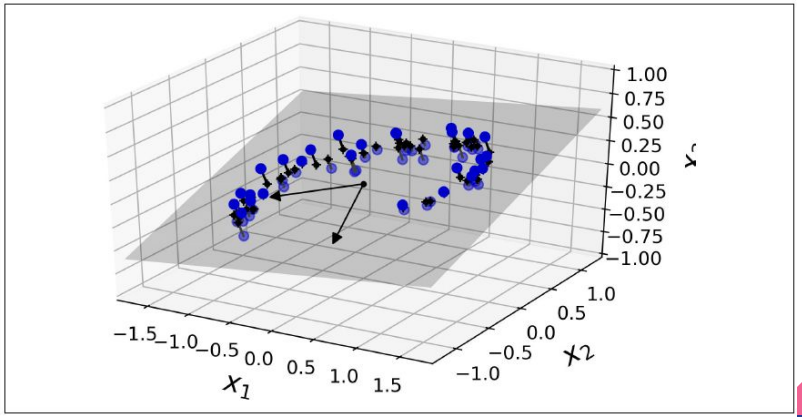

Figure 8-2. A 3D dataset lying close to a 2D subspace

Projection (2)

- All training instances (Figure 8-2) lie close to a plane: this is a **lower-dimensional** (2D) **subspace of the high-dimensional** (3D) **space.**
- Project every training instance perpendicularly onto this subspace to get the new 2D dataset (Figure 8-3). New features are z_1 and z_2.

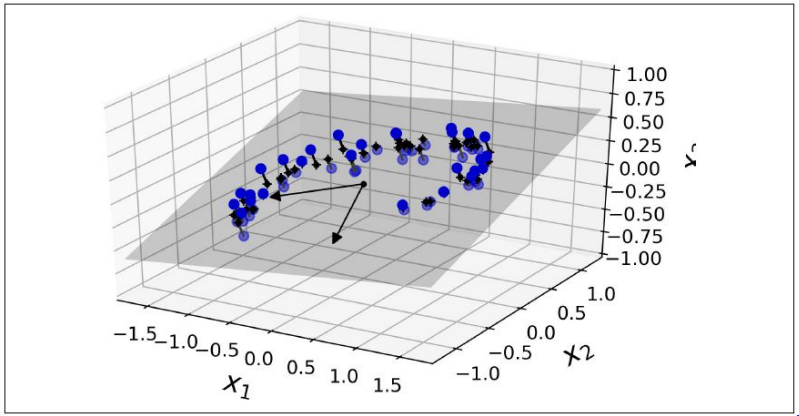

Figure 8-2. A 3D dataset lying close to a 2D subspace

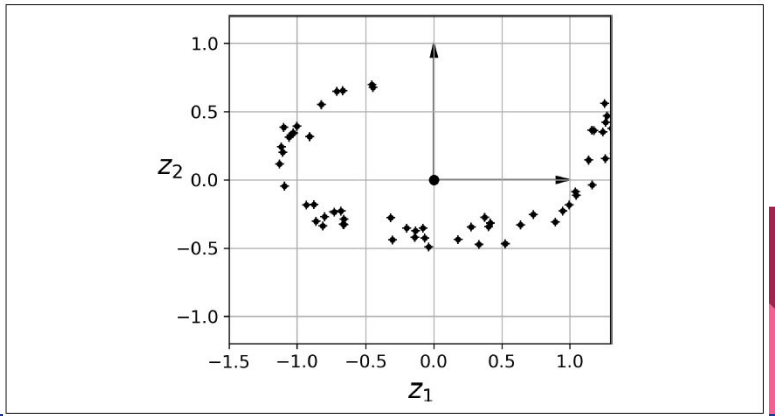

Figure 8-3. The new 2D dataset after projection

Projection (3)

In many cases the subspace may twist and turn, e.g. the *Swiss roll* example.

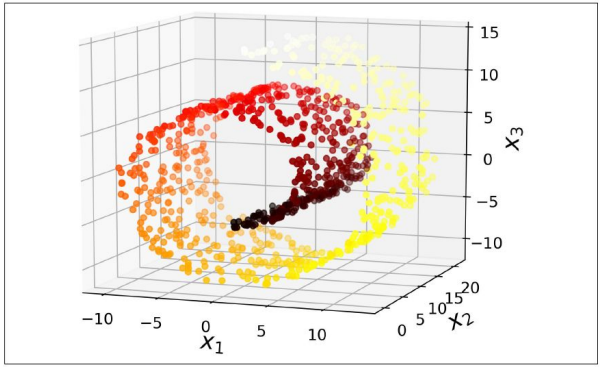

Figure 8-4. Swiss roll dataset

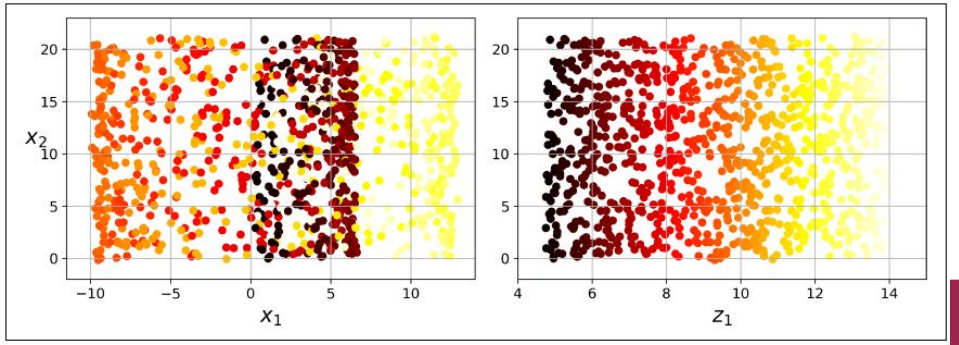

Figure 8-5. Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right)

Manifold Learning (1)

- The *Swiss roll* is an example of a 2D manifold.
- 2D manifold is a 2D shape that can be bent and twisted in a higher-dimensional space.
- A *d*-dimensional manifold is a part of an *n*-dimensional space (where $d < n$) that locally resembles a *d*-dimensional hyperplane.

- Many dimensionality reduction algorithms work by modeling the manifold on which the training instances lie; this is called **Manifold Learning**.
- **manifold assumption** or **manifold hypothesis**: most real-world high-dimensional datasets lie close to a much lower-dimensional manifold.

Manifold Learning (2)

- All handwritten digit images have some similarities. Made of connected lines, the borders are white, they are more or less centered…
- The degrees of freedom (DoF) available to create a digit image are dramatically lower than the DoF to generate any random image you wanted. So, this tends to squeeze the dataset into a lower dimensional manifold.

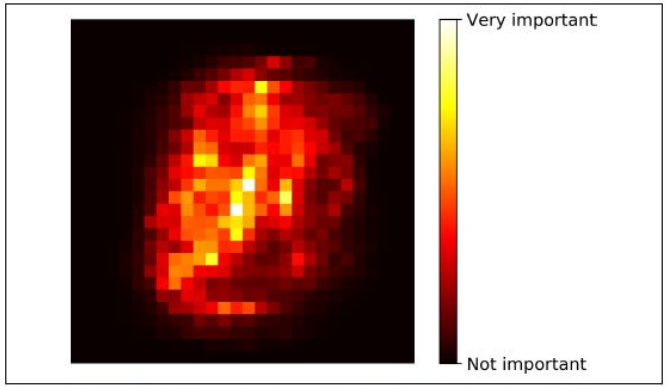

Figure 7-6. MNIST pixel importance (according to a Random Forest classifier)

Manifold Learning (3)

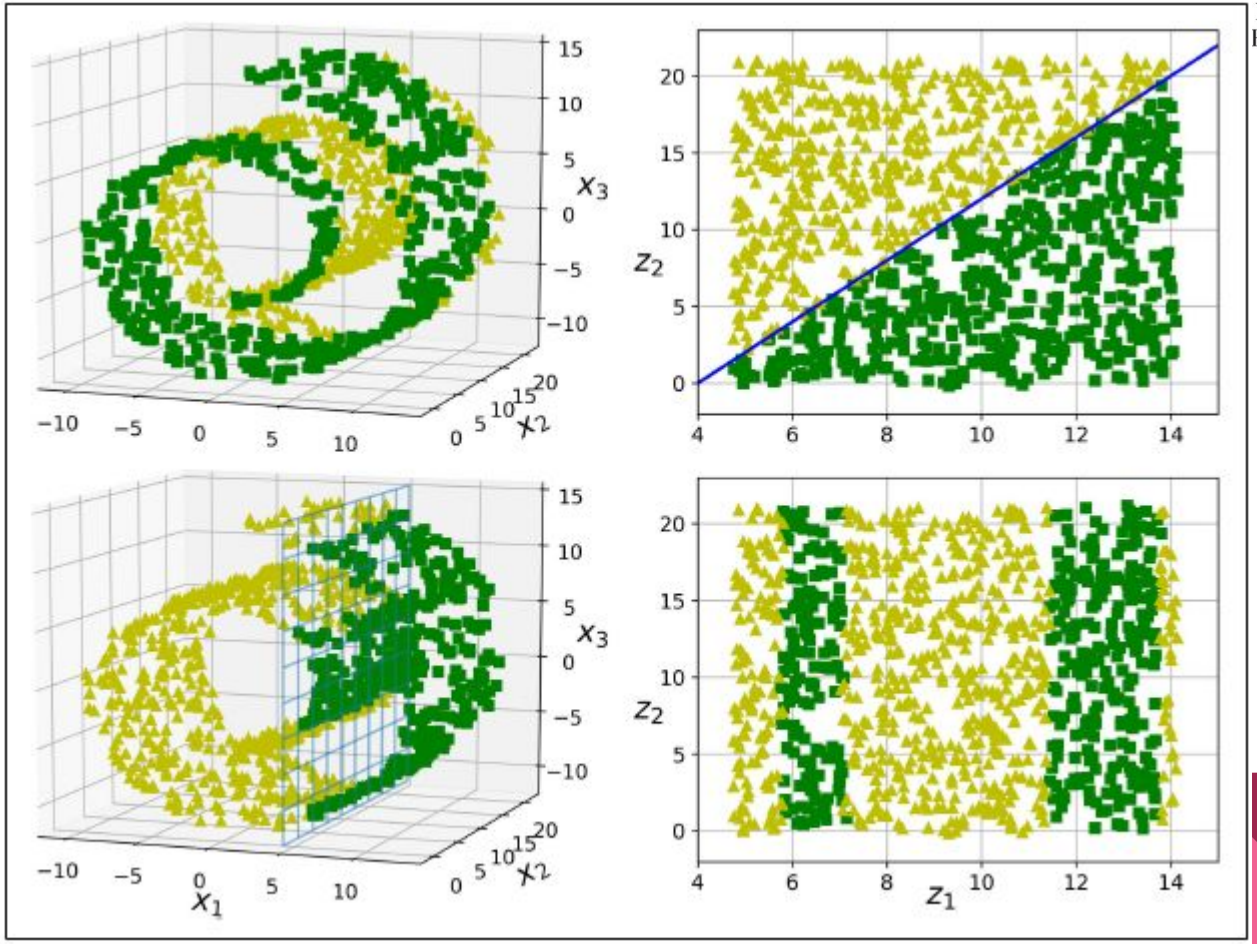

Figure 8-6. The decision boundary may not always be simpler with lower dimensions

Principal Component Analysis (PCA)

The most popular dimensionality reduction algorithm

Principal Component Analysis (PCA)

Identify first the hyperplane that lies closest to the data, then project the data onto it.

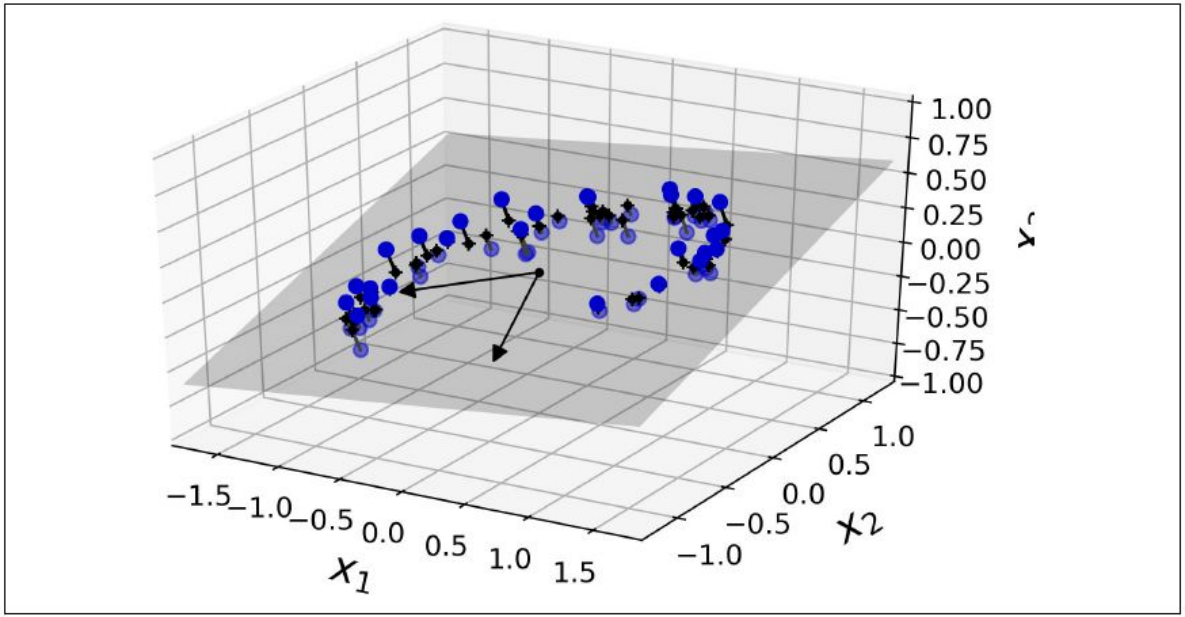

Figure 8-2. A 3D dataset lying close to a 2D subspace

PCA : Preserving the Variance (1)

Choosing the right hyperplane.
(solid line preserves the maximum variance)

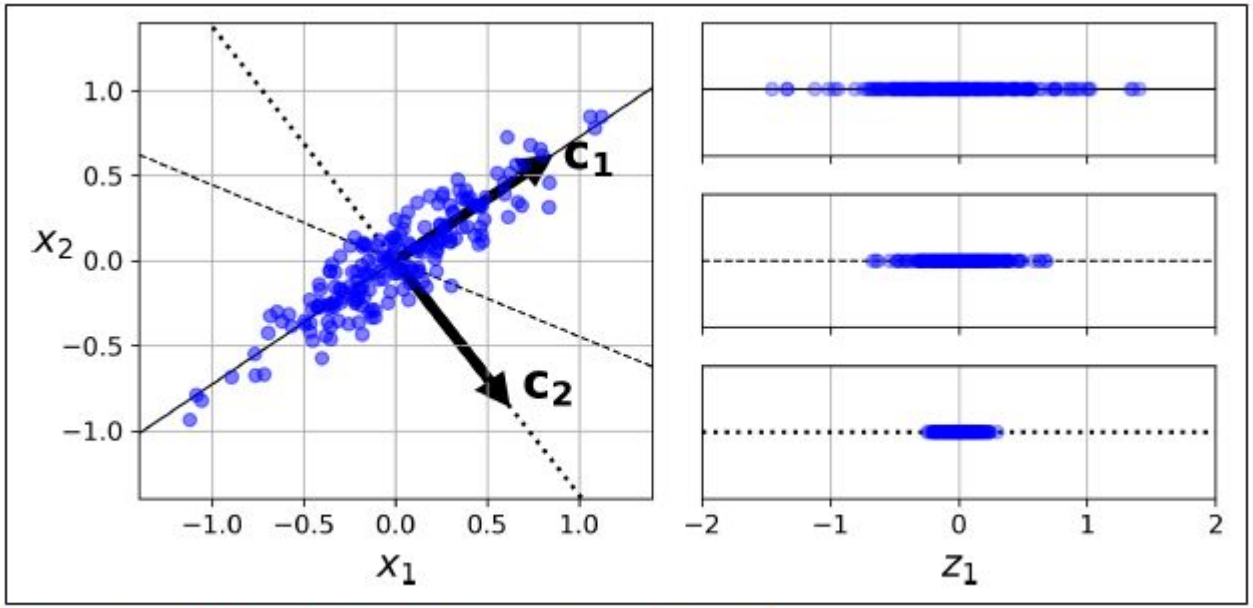

Figure 8-7. Selecting the subspace onto which to project

PCA : Preserving the Variance (2)

- Select the axis that preserves the maximum amount of variance, as it will most likely lose less information than the other projections
- The axis that minimizes the mean squared distance between the original dataset and its projection onto that axis.

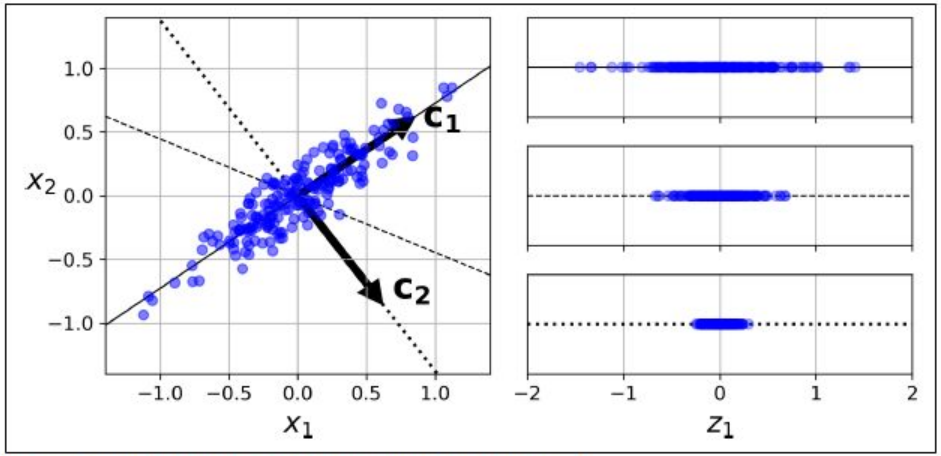

Figure 8-7. Selecting the subspace onto which to project

PCA : Principal Components (1)

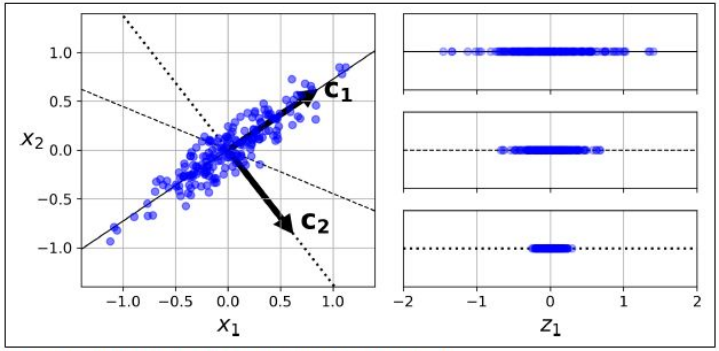

Figure 8-7. Selecting the subspace onto which to project

- PCA identifies the axis that accounts for the largest amount of variance in the training set.
- In given 2D example, it also finds a second axis, orthogonal to the
- first one.
- If it were a higher-dimensional dataset, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on—as many axes as the number of dimensions in the dataset.
- **The unit vector** that defines the i^{th} axis is called the i^{th} ***principal component*** (PC). In Figure 8-7, the 1^{st} PC is c_1 and the 2^{nd} PC is c_2.

PCA : Principal Components (2)

The direction of the principal components is not stable: if you perturb the training set slightly and run PCA again, some of the new PCs may point in the opposite direction of the original PCs. However, they will generally still lie on the same axes. In some cases, a pair of PCs may even rotate or swap, but the plane they define will generally remain the same.

PCA : Principal Components (3)

Singular Value Decomposition (SVD) that can decompose the training set matrix X into the matrix multiplication of three matrices $U \Sigma V^T$, where **V** contains all the principal components.

Equation 8-1. Principal components matrix

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

```
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt.T[:, 0]
c2 = Vt.T[:, 1]
```

PCA : Projecting down to *d* dimensions

Equation 8-2. Projecting the training set down to d dimensions

$$\mathbf{X}_{d\text{-proj}} = \mathbf{X}\mathbf{W}_d$$

To project the training set onto the plane defined by the **first two principal components**:

```
W2 = Vt.T[:, :2]
X2D = X_centered.dot(W2)
```

Using Scikit-Learn

PCA

```
from sklearn.decomposition import PCA

pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
```

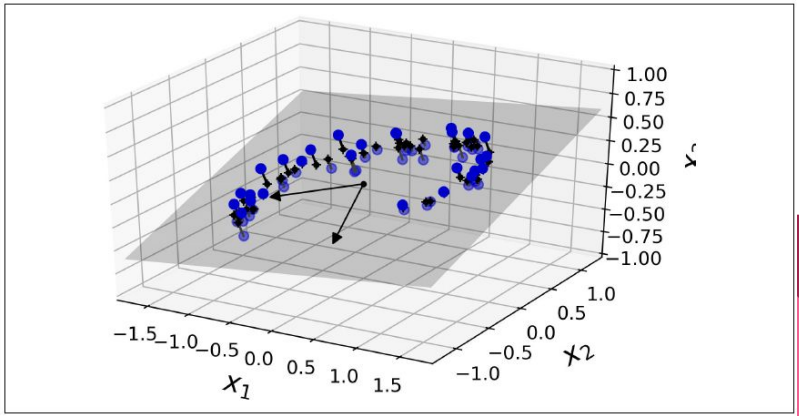

Figure 8-2. A 3D dataset lying close to a 2D subspace

Explained Variance Ratio (def: variance)

- *Explained variance ratio* of each principal component :
  PCA.**explained_variance_ratio_**
  The proportion of the dataset's variance that lies along the axis of each principal component.

```
>>> pca.explained_variance_ratio_
array([0.84248607, 0.14631839])
```

- 84.2% of the dataset's variance lies along the first axis, and 14.6% lies along the second axis. (Less than 1.2% for the third axis. So, it probably carries little information.)

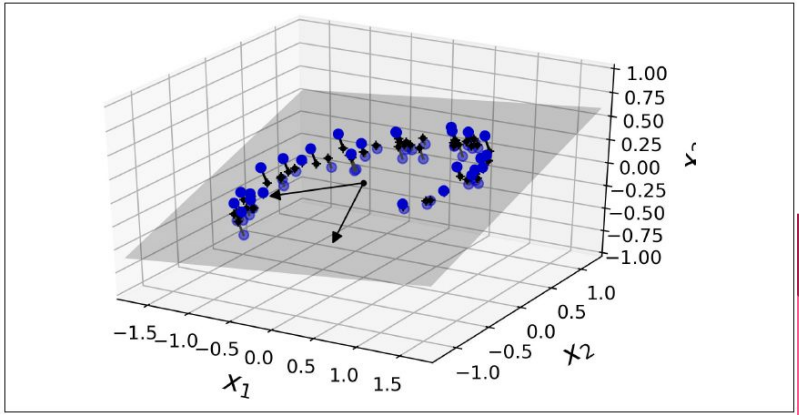

Figure 8-2. A 3D dataset lying close to a 2D subspace

Choosing the Right Number of Dimensions (1)

- Instead of arbitrarily choosing the number of dimensions to reduce down to, it is generally preferable to choose the number of dimensions that **add up to a sufficiently large portion of the variance** (e.g., 95%).
- Example : the minimum number of dimensions required to preserve 95% of the training set's variance:

```
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
```

- PCA.**n_components** :

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

Choosing the Right Number of Dimensions (2)

Plotting **cumsum** to select / decide on / justify the number of dimensions.

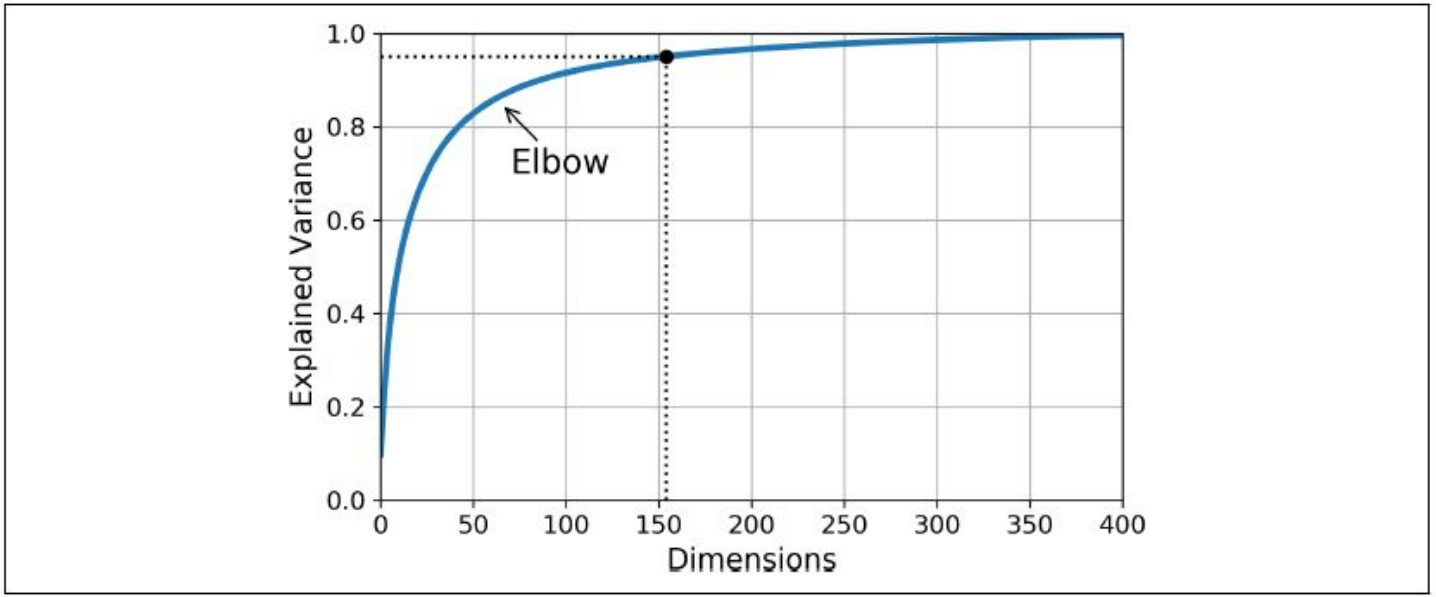

Figure 8-8. Explained variance as a function of the number of dimensions

PCA for Compression (1)

- After dimensionality reduction, the training set takes up much less space.
  - MNIST (preserving 95% if its variance) : 784 features → 150 features.
  - Speeds up classification algorithms tremendously.
- Also possible to decompress the reduced dataset back to 784 dimensions by applying the inverse transformation of the PCA projection.
  - Not getting the original data due to loss of a bit of information (within the dropped variance).
- **Reconstruction error**: the mean squared distance between the original data and the reconstructed data (compressed and then decompressed).

PCA for Compression (2)

Equation 8-3. PCA inverse transformation, back to the original number of dimensions

$$\mathbf{X}_{\text{recovered}} = \mathbf{X}_{d\text{-proj}}\mathbf{W}_d^T$$

- MNIST :
  784 features → (fit_transform) → 154 features → (inverse_transform) → 784 features.

```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

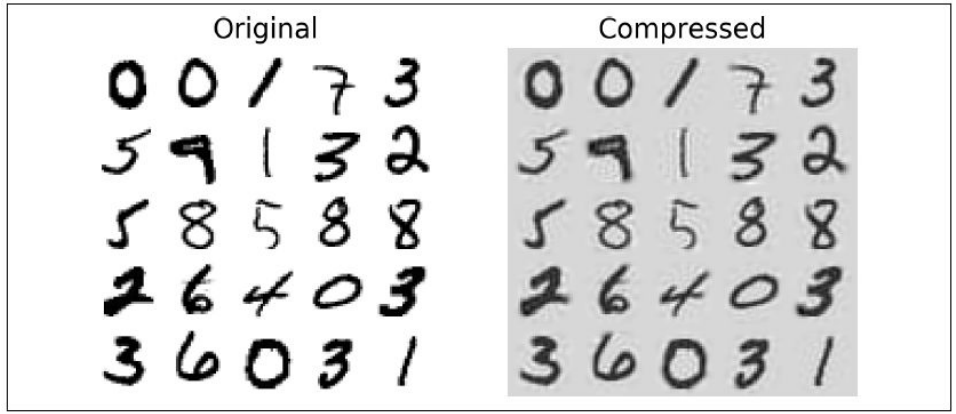

Figure 8-9. MNIST compression preserving 95% of the variance

Randomized PCA : PCA.svd_solver

Based on stochastic algorithm called Randomized PCA that quickly finds an approximation of the first *d* principal components.

Computational complexity $O(m \times d^2) + O(d^3)$ vs. $O(m \times n^2) + O(n^3)$
Randomized PCA by default if *m* or *n* is greater than 500 and *d* is less than 80% of *m* or *n*.

```
rnd_pca = PCA(n_components=154, svd_solver="randomized")
X_reduced = rnd_pca.fit_transform(X_train)
```

Incremental PCA

- To this point, given PCA implementations required the whole training set to fit in memory.
- **Incremental PCA (IPCA)** algorithms: split the training set into mini-batches and feed an IPCA algorithm one mini-batch at a time.
  - Useful for large training sets;
  - PCA online (getting new instances at runtime)
- IncrementalPCA / partial_fit :

```
from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

- / fit :

```
X_mm = np.memmap(filename, dtype="float32", mode="readonly", shape=(m, n))

batch_size = m // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mm)
```

Kernel PCA (1)

- Kernel trick, a mathematical technique that implicitly maps instances into a very high-dimensional space (called the feature space), enabling nonlinear classification and regression (e.g. with Support Vector Machines).
- Linear decision boundary in the high-dimensional feature space corresponds to a complex nonlinear decision boundary in the original space.
- So, also for Kernel PCA or kPCA.
  - Good at preserving clusters of instances after projection, or sometimes even unrolling datasets that lie close to a twisted manifold.

Kernel PCA (2)

```
from sklearn.decomposition import KernelPCA

rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.04)
X_reduced = rbf_pca.fit_transform(X)
```

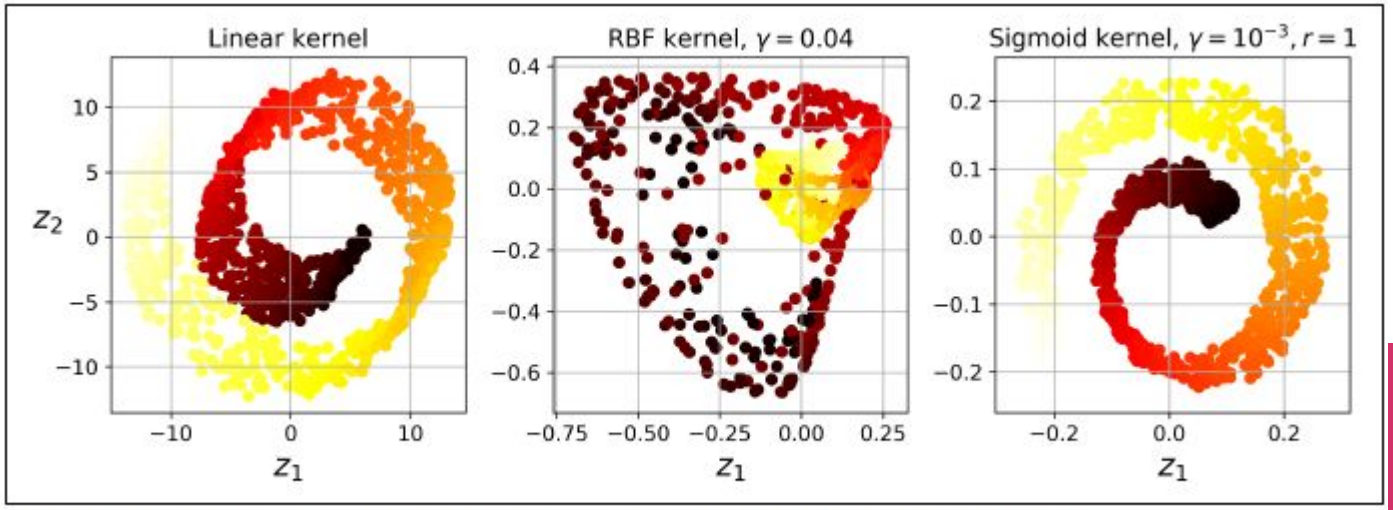

Figure 8-10. Swiss roll reduced to 2D using kPCA with various kernels

Selecting a Kernel and Tuning Hyperparameters (1)

- kPCA is an unsupervised learning algorithm, thus there is no obvious performance measure to help you select the best kernel and hyperparameter values.
- Dimensionality reduction is often a preparation step for a supervised learning task (e.g., classification), so grid search can be used to select the kernel and hyperparameters that lead to the best performance on that task.

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

clf = Pipeline([
        ("kpca", KernelPCA(n_components=2)),
        ("log_reg", LogisticRegression())
    ])

param_grid = [{
        "kpca__gamma": np.linspace(0.03, 0.05, 10),
        "kpca__kernel": ["rbf", "sigmoid"]
    }]

grid_search = GridSearchCV(clf, param_grid, cv=3)
grid_search.fit(X, y)
```

```
>>> print(grid_search.best_params_)
{'kpca__gamma': 0.043333333333333335, 'kpca__kernel': 'rbf'}
```

Selecting a Kernel and Tuning Hyperparameters (2)

- Another approach, to select the kernel and hyperparameters that yield the lowest **reconstruction error**.
  - However, reconstruction is not as easy as with linear PCA.

```
rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.0433,
                    fit_inverse_transform=True)
X_reduced = rbf_pca.fit_transform(X)
X_preimage = rbf_pca.inverse_transform(X_reduced)
```

```
>>> from sklearn.metrics import mean_squared_error
>>> mean_squared_error(X, X_preimage)
32.786308795766132
```

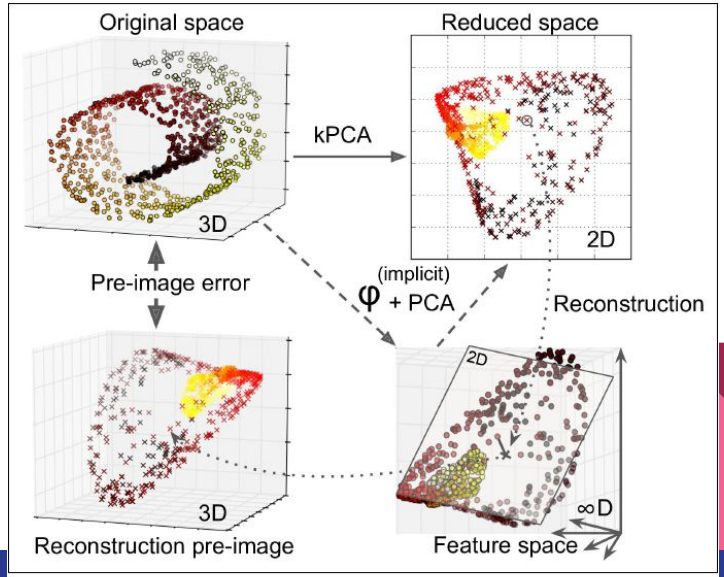

Figure 8-11. Kernel PCA and the reconstruction pre-image error

LLE : Locally Linear Embedding (1)

- A powerful nonlinear dimensionality reduction (NLDR) technique, which does not rely on projections (like the presented before).
- First measures how each training instance linearly relates to its closest neighbors (c.n.), and then looks for a low-dimensional representation of the training set where these local relationships are best preserved.
  - particularly good at unrolling twisted manifolds, especially when there is not too much noise.

```
from sklearn.manifold import LocallyLinearEmbedding

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)
X_reduced = lle.fit_transform(X)
```

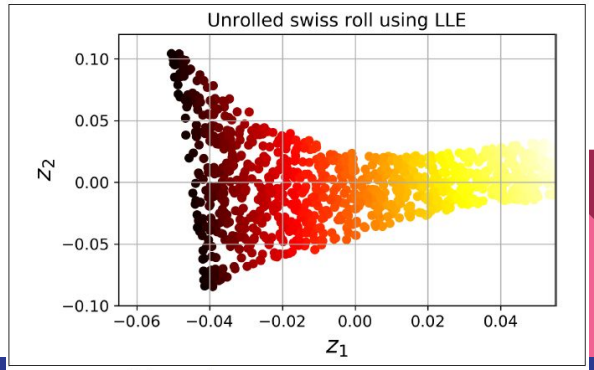

Figure 8-12. Unrolled Swiss roll using LLE

LLE : Locally Linear Embedding (2)

Equation 8-4. LLE step 1: linearly modeling local relationships

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{x}^{(i)} - \sum_{j=1}^{m} w_{i,j} \mathbf{x}^{(j)} \right)^2$$

$$\text{subject to} \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^{m} w_{i,j} = 1 & \text{for } i = 1, 2, \cdots, m \end{cases}$$

Equation 8-5. LLE step 2: reducing dimensionality while preserving relationships

$$\widehat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^{m} \left(\mathbf{z}^{(i)} - \sum_{j=1}^{m} \widehat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$

```
from sklearn.manifold import LocallyLinearEmbedding

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)
X_reduced = lle.fit_transform(X)
```

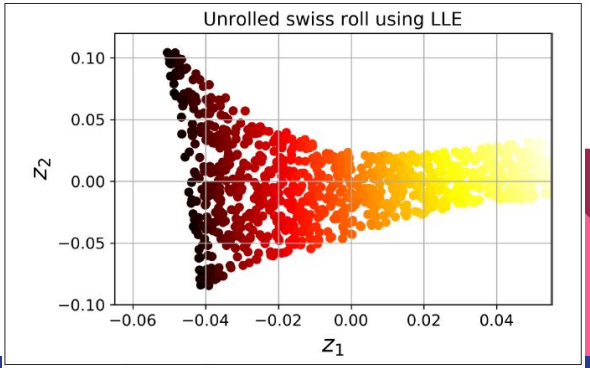

Figure 8-12. Unrolled Swiss roll using LLE

Other Dimensionality Reduction Techniques

- Multidimensional Scaling (MDS) reduces dimensionality while trying to preserve the distances between the instances.
- Isomap creates a graph by connecting each instance to its nearest neighbors, then reduces dimensionality while trying to preserve the geodesic distances between the instances.
- t-Distributed Stochastic Neighbor Embedding (t-SNE) reduces dimensionality
- while trying to keep similar instances close and dissimilar instances apart. It is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space (e.g., to visualize the MNIST images in 2D).
- …

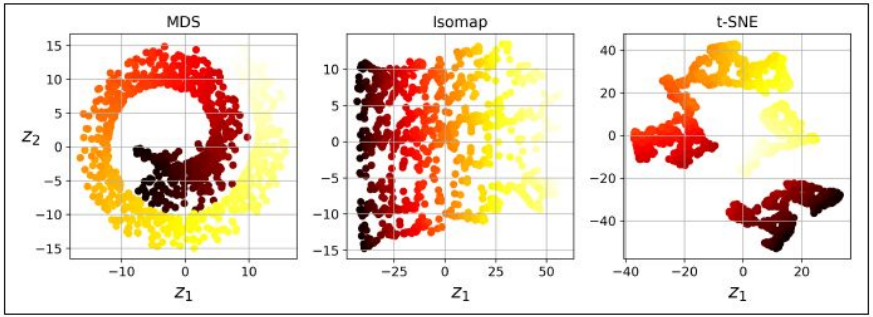

Figure 8-13. Reducing the Swiss roll to 2D using various techniques

Examples

“Integration of Principal Component Analysis with AHP-QFD for Improved Product Design Decision-Making”,

“PCA Analysis of Resource Availability as One of the Inputs in the Process of Estimating the Length of Assembly Time for Complex Products”,

“Maintaining Accuracy While Reducing Effort in Online Decision Making: A New Quantitative Approach for Multi-Attribute Decision Problems Based on Principal Component Analysis”

Takeaways

- Dimensionality reduction for making intractable problems into tractable ones.
- Ways to select principal components, new features.
- Metrics to assess performance.